MCPify Roadmap

> Build software that becomes AI-operable automatically.

This roadmap prioritizes: 1. Core functionality first 2. Demo impact early 3. Fast iteration 4. Future scalability 5. Real ecosystem usefulness

The goal is to first create a working AI-enablement pipeline before expanding into advanced semantic understanding and enterprise infrastructure.

Guiding Principles

Prioritize:

- visible progress
- demoable features
- automation
- usability
- architecture clarity

Avoid early:

- overengineering
- complex infrastructure
- distributed systems
- enterprise abstractions
- premature optimization

PHASE 0 — Foundation Setup

Goal

Create clean project architecture and development environment.

Priority

CRITICAL

Tasks

Repository Setup

- monorepo structure
- package separation
- TypeScript configuration
- linting
- formatting
- shared configs

Basic CLI Setup

Command structure:

```
npx mcpify
```

Using:

- commander.js
- chalk
- ora

Project Structure

```
mcpify/
├── packages/
│   ├── cli/
│   ├── backend-analyzer/
│   ├── frontend-analyzer/
│   ├── schema-engine/
│   ├── mcp-generator/
│   ├── security/
│   └── templates/
```

Deliverables

- working monorepo

- CLI scaffold
- package communication
- local development workflow

PHASE 1 — Backend → MCP MVP

Goal

Generate MCP tools from backend code automatically.

Priority

HIGHEST

This is the first real usable product.

Features

1. Backend Function Scanner

Scan:

- exported functions
- async functions
- TypeScript types

Using:

- ts-morph
- TypeScript Compiler API

---

2. Type Extraction

Extract:

- parameter types
- return types
- comments/docstrings

---

3. Schema Generation

Generate:

- zod schemas
- validation layers

Example:

```
function getWeather(city: string)
```

→

```
z.object({
  city: z.string()
})
```

---

4. MCP Tool Generator

Generate:

- server.tool(...)
- tool metadata
- handlers

---

5. Template System

Generate:

- runnable MCP server
- configuration files
- package.json
- starter templates

CLI

```
npx mcpify ./src
```

Deliverables

User Can:

- scan backend
- generate MCP server
- connect to Claude/Codex

Success Criteria

A developer can: 1. point MCPify at backend code 2. instantly get AI tools

WITHOUT writing MCP manually.

PHASE 2 — OpenAPI → MCP

Goal

Convert APIs directly into MCP servers.

Priority

VERY HIGH

This massively expands usability.

Features

OpenAPI Parser

Support:

- Swagger
- OpenAPI 3.x

Using:

- swagger-parser

Endpoint Extraction

Extract:

- routes
- parameters
- request bodies
- auth metadata

MCP Conversion

Generate:

- tools
- schemas
- examples
- endpoint wrappers

CLI

```
npx mcpify swagger.json
```

Deliverables

User Can:

- convert existing SaaS APIs into MCP instantly

Why This Matters

Huge adoption potential because:

- companies already have APIs
- no source-code access required

PHASE 3 — Frontend Action Extraction

Goal

Make frontend applications AI-operable.

Priority

VERY HIGH

This is the major differentiator.

Features

Frontend Scanner

Support:

- React
- Next.js

Detect:

- buttons
- forms
- routes
- actions
- handlers

Event Extraction

Extract:

- onClick
- form submit handlers
- navigation actions

Semantic Action Generation

Convert:

```
<button onClick={checkout}>
```

into:

```
checkoutCart()
```

Route Understanding

Detect:

- pages
- workflows
- navigation structure

CLI

```
npx mcpify frontend
```

Deliverables

AI Can:

- understand frontend actions
- trigger workflows
- interact with app logic

Success Criteria

A frontend app becomes AI-operable without browser automation.

PHASE 4 — Security & Permissions Layer

Goal

Prevent unsafe AI operations.

Priority

HIGH

Critical for trust and real usage.

Features

Dangerous Action Detection

Detect:

- delete operations
- shell execution
- env access
- unrestricted writes

Tool Classification

Classify:

- SAFE
- CONFIRMATION_REQUIRED
- BLOCKED

Permission Rules

Support:

- RBAC
- scopes
- allowlists
- deny rules

Audit Mode

```
npx mcpify --audit
```

Deliverables

Generated MCP tools become safer by default.

PHASE 5 — Workflow Engine

Goal

Detect multi-step workflows automatically.

Priority

MEDIUM-HIGH

Moves MCPify from tool generation to intelligent system understanding.

Features

Workflow Detection

Identify:

- login flows
- checkout flows
- onboarding flows

Workflow Graph Generation

Generate:

- workflow DAGs
- execution chains

- dependencies

Workflow Tools

Generate:

```
purchaseWorkflow()
```

instead of only primitive actions.

Deliverables

AI agents can execute higher-level business operations.

PHASE 6 — AI Metadata Enhancement

Goal

Improve generated tools using LLMs.

Priority

MEDIUM

Improves usability and polish.

Features

AI Descriptions

Generate:

- descriptions
- examples
- summaries

Naming Cleanup

Convert:

- unclear functions
- short names
- ambiguous parameters

into understandable tool metadata.

Optional AI Layer

Should work without requiring AI APIs.

Deliverables

Generated tools become much more agent-friendly.

PHASE 7 — AGENTS.md Generator

Goal

Generate repository context for AI coding agents.

Priority

MEDIUM

Useful ecosystem feature.

Features

Generate:

- architecture overview
- commands
- workflows
- conventions

- project summaries

CLI

```
npx mcpify --agents
```

Deliverables

Better AI coding agent performance on repositories.

PHASE 8 — Database Intelligence

Goal

Generate semantic DB tools.

Priority

MEDIUM

Features

Support:

- Prisma
- Drizzle
- PostgreSQL

Generate:

- query tools
- filtered access
- semantic operations

Example

```
getOrdersByStatus()
```

Deliverables

AI can reason over application data safely.

PHASE 9 — Event System Integration

Goal

Enable event-driven AI systems.

Priority

LOWER

Features

Support:

- Kafka
- RabbitMQ
- Webhooks

Generate:

- subscriptions
- event handlers
- reactive workflows

Deliverables

AI agents become event-aware.

PHASE 10 — Knowledge Graph Engine

Goal

Build semantic application understanding.

Priority

ADVANCED

Features

Build graph of:

- APIs
- UI
- workflows
- services
- permissions

Purpose

Allow AI agents to:

- reason globally
- understand dependencies
- navigate applications contextually

PHASE 11 — Self-Updating Synchronization

Goal

Prevent MCP drift.

Priority

ADVANCED

Features

Watch:

- source code
- APIs
- schemas

Auto-regenerate:

- tools
- workflows
- docs

Deliverables

Always-synchronized AI layer.

PHASE 12 — AI Simulation & Validation

Goal

Automatically test generated AI systems.

Priority

ADVANCED

Features

Test:

- prompt injection
- invalid parameters
- permission bypass
- unsafe workflows

CLI

```
npx mcpify simulate
```

Deliverables

Production-grade AI validation pipeline.

Recommended Immediate Build Order

START HERE

Week 1

- monorepo
- CLI
- backend scanner

Week 2

- schema generation
- MCP generator
- runnable output

Week 3

- OpenAPI support
- frontend extraction MVP

Week 4

- semantic frontend actions
- security layer
- demo polishing

Most Important Features

If time becomes limited:

MUST HAVE

- backend → MCP
- OpenAPI → MCP
- frontend action extraction

These create the strongest demo.

SHOULD HAVE

- security warnings
- semantic naming
- workflow detection

NICE TO HAVE

- knowledge graph
- AI simulations
- self-updating sync

Final Goal

Transform MCPify from:

- MCP scaffolding tool

into:

- AI enablement infrastructure layer

for the next generation of AI-operable software systems.